

TEMPO: Taxonomy-Enforced Multi-Robot Planning and Optimization via Large Language Models

KAAVISH PROJECT PROPOSAL

By

Abdullah Shaikh	(09245@st.habib.edu.pk)
Salman Kamal	(sk08996@st.habib.edu.pk)
Taha Zahid	(tz09220@st.habib.edu.pk)
Zain Naqi	(09224@st.habib.edu.pk)

September 27, 2026



In partial fulfillment of the requirement for
Bachelor of Science
Computer Science

DHANANI SCHOOL OF SCIENCE AND ENGINEERING

HABIB UNIVERSITY

FALL 2026

Copyright © 2026 Habib University

1 Abstract

Robots are increasingly used to perform complex tasks in warehouses, hospitals, construction sites, and disaster response scenarios. In many of these situations, a single robot is not enough. A task might be too large, require different types of equipment, or need to be finished quickly. This is where multi-robot systems come in, where teams of robots work together toward a shared goal. Coordinating that team, though, is not simple. Deciding which robot should do which task, in what order, and by when is a hard problem in itself. This problem is called Multi-Robot Task Allocation, or MRTA.

For a long time, MRTA systems were programmed for specific scenarios. A team of robots built to sort packages in a warehouse could do exactly that, but nothing more. If the mission changed or an unexpected situation came up, the system had no way to handle it. The robots could only do what they were explicitly programmed for.

Large Language Models changed what was possible here. These models, trained on large amounts of text, can read natural language instructions, reason about goals, and break a complex task into smaller steps. Researchers started using them in multi-robot systems so that a human could describe a mission in plain language and have the robots figure out how to handle it. This opened up a lot of new possibilities. The problem is that most of these systems also use the LLM to decide which robot gets which task. That step, deciding who does what, is a mathematical problem. It requires finding an assignment that satisfies constraints around robot capabilities, travel distances, timing, and whether some tasks need to be done simultaneously by multiple robots. LLMs do not handle this step well since they are not mathematical solvers. Research has shown that when asked to make these decisions, they tend to produce assignments that are unbalanced, infeasible, or far from optimal, even when given the chance to review and correct their own output.

We present TEMPO (Taxonomy-Enforced Multi-Robot Planning and Optimization), which takes a different approach. The LLM is used only for the parts of the problem it handles well: reading the mission description, breaking it into subtasks, figuring out which subtasks depend on which others, and determining what type of coordination problem the mission represents. Once the problem type is identified, a dedicated mathematical solver handles the actual assignment of robots to tasks. Different types of problems have different mathematical structures, and there are well-established algorithms for each. Using the right solver for each problem type produces better results than asking the LLM to figure everything out.

We test TEMPO in a simulated environment using a mixed fleet of wheeled ground robots, robotic arms, etc, all with different capabilities. The simulation covers seven different problem types, from straightforward cases where robots independently handle separate tasks to more involved scenarios where groups of robots need to coordinate in a particular order. We compare the results against systems that leave the allocation step to the LLM, using measures like total mission time, travel distance, token cost and how far the solution is from the best possible assignment.

The aim is to show that keeping task allocation as a mathematical problem, and using the LLM only for language understanding, leads to more dependable outcomes. If this holds up, it could make multi-robot systems more practical in settings where task assignment is critical for the success and efficiency of the task.

2 Problem definition

Multi-Robot Task Allocation (MRTA) is the problem of computing an assignment from a set of tasks $\mathcal{T}$ to a team of heterogeneous robots $\mathcal{R}$ such that a global objective, typically total utility or mission completion time, is improved subject to physical and operational constraints [1]. The difficulty of this problem is not the same in every scenario; it depends on the structure of the robots, the tasks, and the dependencies among them.

Gerkey and Mataric [1] formalized MRTA through a three-axis taxonomy. The first axis distinguishes Single-Task robots (ST), which execute at most one task at a time, from Multi-Task robots (MT). The second distinguishes Single-Robot tasks (SR), which require one robot, from Multi-Robot tasks (MR), which require a group of robots with complementary capabilities. The third distinguishes Instantaneous Assignment (IA), where assignment is made using information available at that moment, from Time-Extended Assignment (TA), where a schedule is built over a planning period. These combinations lead to different allocation and scheduling problems.

Korsah et al. [2] identified a limitation in this taxonomy: it assumes that the value of assigning a robot to a task is independent of other assignments. Their extended taxonomy, iTax, adds a dependency dimension. In No-Dependency (ND) problems, task assignments are independent. In In-Schedule Dependency (ID) problems, a robot's cost depends on the sequence of its own tasks, such as travel distance or total travel time. In Cross-Schedule Dependency (XD) problems, the schedules of different robots are linked through requirements such as precedence, time synchronization, or coalition work. TEMPO targets seven subclasses: ND[ST-SR-IA], ND[ST-SR-TA], ID[ST-SR-TA], XD[ST-SR-IA], XD[ST-SR-TA], XD[ST-MR-IA], and XD[ST-MR-TA]. Crucially, while the first two subclasses admit polynomial-time formulations, the remaining five subclasses are NP-hard, requiring integer programming, heuristics, or metaheuristics.

Traditional MRTA systems require missions to be given as structured inputs, limiting them to tasks they were explicitly programmed to handle. LLMs remove this constraint: a user can describe any mission in natural language, and the LLM can interpret it and break it into subtasks. This makes LLM-based MRTA systems capable of handling tasks without task-specific programming, which is a significant step toward general-purpose robot coordination. However, many of these systems also use the LLM to decide which robot should perform each subtask. Task allocation requires combinatorial reasoning over numerical constraints, and LLMs are not suited for this; they can produce suboptimal or infeasible assignments in these cases [3].

TEMPO addresses two related problems. First, how can a natural-language mission be interpreted, broken into a structured set of subtasks with dependencies, and classified into one of the iTax subclasses? Second, given that classification, which optimisation method should be used and how should its result be dispatched to a heterogeneous robot fleet? The key idea is that language understanding and combinatorial optimisation require different tools. The LLM is used for mission interpretation, while a specialised solver performs task allocation.

3 Social relevance

Multi-robot systems can support work that is important to people. In hospitals, they can move medicine and equipment. In disasters, they can enter unsafe areas to search for survivors or deliver supplies. In warehouses and farms, they can help move goods and monitor large areas. These uses can save time, reduce physical workload, and keep people away from dangerous tasks.

For these benefits to be useful, the robots must be given suitable work and must work together safely. A poor decision can delay urgent supplies, leave an important task unfinished, or send the wrong robot to a job. TEMPO supports more reliable coordination of robot teams. In the long term, this can help organisations use robots more safely and effectively in services that affect the public.

4 Originality/Novelty

Classical MRTA systems use hand-crafted allocation methods, such as auctions, greedy matching, or fixed optimisation formulations. Systems such as ALLIANCE, BLE, and MURDOCH are well studied and give predictable results, but they require structured mission inputs and are built around a fixed allocation method [1]. They cannot interpret a mission given in natural language or adapt their allocation method when the problem structure changes.

LLM-based multi-robot systems address the natural-language limitation, but often use the LLM for both mission interpretation and task allocation. This is a problem because allocation must satisfy capability, timing, and dependency constraints. LLMs can understand a mission, but they are not designed to solve these combinatorial problems. As a result, they can produce infeasible or suboptimal allocations, even when asked to check their own answers [3].

Recent hybrid systems pair an LLM with a downstream solver. For example, LiP-LLM extracts constraints from text and solves them using a linear program [4]. While this avoids using the LLM for arithmetic, such systems are tied to a single, fixed optimisation model. They cannot identify the underlying structure of a general multi-robot problem or route it to an appropriate solver across different dependency classes.

TEMPO adds a taxonomy-classification layer between the LLM and the optimisation engine. When unguided language models attempt to schedule tasks with complex timing and synchronization constraints, they routinely misalign robot schedules, producing execution timelines that are “not quite my tempo”. TEMPO addresses this by using the LLM to convert a natural-language mission into a task structure and classify it into one of seven iTax subclasses [2]. The system then routes the mission to a solver designed for that class. Depending on the subclass, this may be an exact method such as the Hungarian algorithm or an integer program, or a metaheuristic such as a genetic algorithm where exact methods become intractable. This approach combines the flexibility of natural-language missions with an allocation method that matches the problem structure.

5 CS contribution

This project draws on several areas of Computer Science. The first is artificial intelligence and natural language processing. An LLM-based planning component with retrieval-augmented generation (RAG) will be built to interpret natural-language missions, extract subtasks with their dependencies and capability requirements, and produce a schema-constrained JSON plan. This requires prompt design, retrieval and indexing, structured output generation, and error handling for invalid or incomplete plans.

The second area is algorithms and graph-based modelling. The extracted subtasks will be represented as a Directed Acyclic Graph (DAG), where each node is a subtask and each edge encodes a dependency. The graph will be validated and used to determine task ordering constraints before allocation.

The novel CS contribution of TEMPO is the taxonomy classification layer. Rather than treating all task-allocation problems with a single method, the system uses the LLM to classify each mission into one of seven iTax subclasses and then routes it to a solver matched to that subclass’s mathematical structure. To do this accurately, a second RAG component will be built specifically for taxonomy knowledge, indexing axis definitions, subclass criteria, and worked examples with classification reasoning that the LLM retrieves at inference time. This requires curating the taxonomy knowledge base and validating classification consistency across all seven subclasses.

Combinatorial optimisation is a third CS area the project requires. While the first two subclasses admit polynomial-time solutions, the remaining five subclasses are NP-hard. They map to problem formulations including vehicle routing, integer programming, and coalition formation, which require exact integer programming for small instances or metaheuristic methods such as genetic algorithms for larger ones.

Finally, the project involves robotics software integration. ROS nodes will translate solver outputs into action commands dispatched to a heterogeneous simulated fleet in Gazebo, coordinating execution across robots with different capabilities.

6 Scope and Deliverables

The project will be completed over two academic semesters in a mapped, simulated environment with reliable communication and open-loop execution.

6.1 Semester 1

Mission Set Definition. The team will first survey existing literature to identify benchmark problems comparable to those used in prior work and select at least one natural-language mission per iTax subclass. The mission set will be submitted as part of the Project Plan and SRS on 4 November 2026.

LLM-Based Mission Planning and Classification. In parallel with the solver work below, an LLM and RAG module will be implemented that converts a natural-language mission into a validated JSON task plan with subtasks, dependencies, and an iTax subclass label. Progress will be reported on 4 November 2026 and updated at the meetings on 11 and 18 November 2026.

Task-Allocation Solvers for Four Problem Subclasses. In parallel with the LLM module, solvers will be implemented for ND[ST-SR-IA], ND[ST-SR-TA], ID[ST-SR-TA], and XD[ST-SR-IA]. Each solver must return a feasible assignment satisfying the capability, dependency, and timing constraints for its subclass. Solver implementations will be covered in the SDS submitted on 25 November 2026.

2D Low-Fidelity Simulation. The planning module and four solvers will be validated end-to-end in a 2D simulation on the defined mission set, to be demonstrated at the Prototype Review on 2 December 2026.

6.2 Semester 2

Task-Allocation Solvers for Remaining Three Subclasses. Solvers for XD[ST-SR-TA], XD[ST-MR-IA], and XD[ST-MR-TA] will be implemented and validated, completing coverage across all seven subclasses.

3D Gazebo Environment, ROS Integration, and Evaluation. A 3D Gazebo environment will be built containing tasks for all seven subclasses. The full pipeline will be integrated with ROS 2, with nodes dispatching solver outputs as action commands to the simulated fleet. The system will then be benchmarked against baselines including direct LLM allocation and greedy assignment, reporting classification accuracy, schedule feasibility, mission time, travel distance, solver time, and optimality gap.

7 Feasibility

TEMPO relies on a robotics stack built around ROS 2, Gazebo, MoveIt2, Nav2, and an LLM-based planning layer using Python and NetworkX. The seven iTax subclasses map to well-studied mathematical formulations with established algorithmic solutions. Simpler subclasses are solved with exact methods using mature libraries including SciPy, OR-Tools, and PuLP, while harder NP-hard subclasses follow published implementations using exact or metaheuristic methods. Knowledge resources, including the robot capability registry, skill banks, and world maps, will be authored by the team as structured JSON and YAML files. The seven benchmark missions require no external datasets, the simulation setup eliminates physical hardware needs, and remote API access handles LLM inference without local GPU dependencies.

8 Team dynamics

All four members contribute across the full system, combining complementary backgrounds while developing in unfamiliar areas. Abdullah brings robotics software experience from STRP research on LLM-controlled multi-robot systems, an autonomous vehicle internship at Toyota using ROS and Gazebo, and prior work with RAG pipelines; he will develop further in combinatorial optimisation. Salman brings game-development and AI coursework experience suited to building the simulation environment and heuristic methods, and will expand into ROS/Gazebo and formal solvers. Zain and Taha have the strongest background in algorithms and computational intelligence, leading the taxonomy-to-optimisation mapping and solver design, while building deeper ROS/Gazebo expertise for Semester 2 integration. All members share a foundation in ROS and multi-robot coordination through Mobile Robotics coursework.

9 Tech stack

The system is built on a ROS 2 and Gazebo stack, with MoveIt2 and Nav2 handling arm motion planning and mobile navigation respectively, ros2_control managing robot controllers, and URDF/Xacro and SDF defining robot and world models. The AI planning layer uses Anthropic APIs for LLM inference, with Python and NetworkX handling DAG construction and dependency reasoning. Optimization solvers for the seven iTax subclasses are implemented using SciPy, OR-Tools, and PuLP. Knowledge resources including the robot capability registry, skill banks, and world maps are stored as JSON and YAML files.

10 References

References

- [1] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004. <https://doi.org/10.1177/0278364904045564>.
- [2] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research*, 32(12):1495–1512, 2013. <https://doi.org/10.1177/0278364913496484>.
- [3] Peihan Li, Zijian An, Shams Abrar, and Lifeng Zhou. Large language models for multi-robot systems: A survey. <https://arxiv.org/abs/2502.03814>, 2026.
- [4] Kazuma Obata, Tatsuya Aoki, Takato Horii, Tadahiro Taniguchi, and Takayuki Nagai. Lip-llm: Integrating linear programming and dependency graph with large language models for multi-robot task planning. <https://arxiv.org/abs/2410.21040>, 2024.

Undertaking of Kaavish advisement as an External Supervisor

I hereby affirm that I have read the project details as described on the preceding pages and agree to undertake advisement of this Kaavish project as an External Supervisor. I understand that this role entails the following.

Meeting Meeting the project team regularly, at least once every two weeks, for the entire duration of the Kaavish. The meetings may be held remotely if required.

Advisement Providing supervision and advice to the team in order to ensure steady progress of the project toward its goals.

Liaison Liaising with the Internal Supervisor as required, e.g. to provide feedback or engage in grading.

Other Any other task, depending on availability and suitability, relevant to the Kaavish as communicated by the Internal Supervisor or Kaavish Working Group.

Name: _____

Email: _____

Phone: _____

Designation: _____

Affiliation: _____

Signature: _____